



**AVIGNON**  
UNIVERSITÉ

# Projet Amiiibo

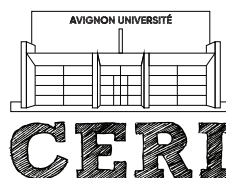
L3 CMI Informatique  
**Angelo Adragna**

**08/04/2025**

**Licence d'Informatique**  
**UE** Application mobile et Ergonomie

**Responsables**  
Mohamed Morchid

**UFR**  
**SCIENCES**  
**TECHNOLOGIES**  
**SANTÉ**



**CENTRE**  
**D'ENSEIGNEMENT**  
**ET DE RECHERCHE**  
**EN INFORMATIQUE**  
[ceri.univ-avignon.fr](http://ceri.univ-avignon.fr)

## Sommaire

|                                                                                                   |           |
|---------------------------------------------------------------------------------------------------|-----------|
| <b>Titre</b>                                                                                      | <b>1</b>  |
| <b>Sommaire</b>                                                                                   | <b>2</b>  |
| <b>1 Introduction</b>                                                                             | <b>4</b>  |
| 1.1 Présentation du sujet                                                                         | 4         |
| 1.2 Objectifs du rapport                                                                          | 4         |
| 1.3 Remarque                                                                                      | 4         |
| <b>2 Contexte de travail</b>                                                                      | <b>4</b>  |
| 2.1 Présentation de l'organisation                                                                | 4         |
| 2.2 Structure et fonctionnement                                                                   | 4         |
| 2.3 Méthodologies employées                                                                       | 5         |
| <b>3 Outils employés (logiciels et autres)</b>                                                    | <b>6</b>  |
| 3.1 Environnement de développement intégré (IDE)                                                  | 6         |
| 3.2 Langage de programmation                                                                      | 6         |
| 3.3 Gestion de la base de données                                                                 | 6         |
| 3.4 Outils de versioning                                                                          | 6         |
| 3.5 Outils d'intelligence artificielle                                                            | 6         |
| 3.6 Outils de traitement d'image                                                                  | 6         |
| 3.7 Utilisation de maquettage                                                                     | 6         |
| 3.8 Tests et débogage                                                                             | 7         |
| <b>4 Modélisation (via un logiciel dédié, use case, diagramme de classes, etc.)</b>               | <b>8</b>  |
| 4.1 Logiciel UML                                                                                  | 8         |
| 4.2 Diagramme de cas d'utilisation                                                                | 8         |
| 4.3 Diagramme de classes                                                                          | 9         |
| <b>5 Difficultés rencontrées</b>                                                                  | <b>10</b> |
| 5.1 Problèmes techniques                                                                          | 10        |
| 5.1.1 Compatibilité et intégration                                                                | 10        |
| 5.1.2 Performance et optimisation                                                                 | 10        |
| 5.2 Problèmes organisationnels                                                                    | 11        |
| 5.2.1 Communication interne                                                                       | 11        |
| 5.2.2 Gestion du temps de chargement                                                              | 11        |
| 5.2.3 Synchronisation                                                                             | 11        |
| <b>6 Solutions disponibles, solutions choisies et justifications (avantages et inconvénients)</b> | <b>12</b> |
| 6.1 Solutions disponibles pour MainActivity                                                       | 12        |
| 6.1.1 ListView                                                                                    | 12        |
| 6.1.2 RecyclerView                                                                                | 12        |
| 6.1.3 Analyse comparative                                                                         | 12        |
| 6.2 Construction des activités                                                                    | 12        |
| 6.2.1 Redéfinir les bases                                                                         | 12        |
| 6.2.2 S'adapter à vos consignes                                                                   | 12        |
| 6.2.3 Analyse comparative                                                                         | 12        |
| 6.3 Structure des classes : 'Amiibo', 'AmiiboGame', 'AmiiboHeader'                                | 13        |
| 6.3.1 Adaptation                                                                                  | 13        |
| 6.3.2 Respect de la consigne                                                                      | 13        |

|          |                                                                    |           |
|----------|--------------------------------------------------------------------|-----------|
| 6.3.3    | Analyse comparative                                                | 13        |
| 6.4      | Menus et barres d'actions                                          | 13        |
| 6.4.1    | Menu de 'MainActivity'                                             | 13        |
| 6.4.2    | Menu de 'GameActivity'                                             | 13        |
| 6.4.3    | Menu et barres de 'HomeActivity', 'RewardActivity', 'HelpActivity' | 13        |
| 6.5      | Récupération des Amiiibo                                           | 13        |
| 6.5.1    | Traitement local                                                   | 13        |
| 6.5.2    | Requête spécifique                                                 | 14        |
| 6.5.3    | Analyse comparative                                                | 14        |
| 6.6      | Initialisation de la sélection dans le 'RecyclerView'              | 14        |
| 6.6.1    | Analyse comparative                                                | 14        |
| 6.7      | Bouton "tout sélectionner/désélectionner"                          | 14        |
| 6.7.1    | Approche intelligente                                              | 14        |
| 6.7.2    | Approche simple                                                    | 14        |
| 6.7.3    | Analyse comparative                                                | 14        |
| <b>7</b> | <b>Utilisabilité (points liés à l'ergonomie traités, etc.)</b>     | <b>15</b> |
| 7.1      | Navigation intuitive                                               | 15        |
| 7.2      | Interaction avec les éléments                                      | 15        |
| 7.3      | Lors d'un partie                                                   | 15        |
| 7.4      | Affichage des données                                              | 16        |
| 7.5      | Tests d'ergonomie                                                  | 16        |
| 7.6      | Gestion des données                                                | 16        |
| 7.7      | Maquette                                                           | 17        |
| 7.8      | Inspirations                                                       | 18        |
| <b>8</b> | <b>Fonctionnalités développées</b>                                 | <b>21</b> |
| 8.1      | Présentation des fonctionnalités                                   | 21        |
| 8.1.1    | Changer de thème                                                   | 21        |
| 8.1.2    | Gestion de la musique                                              | 21        |
| 8.1.3    | Changer d'icônes                                                   | 21        |
| 8.1.4    | Choisir son nom d'utilisateur                                      | 21        |
| 8.1.5    | Suivi statistique                                                  | 21        |
| 8.1.6    | Une motivation pour jouer                                          | 21        |
| 8.1.7    | Supprimer la base de données                                       | 21        |
| 8.1.8    | Lancer un quiz                                                     | 21        |
| 8.1.9    | Choisir le type de Question                                        | 22        |
| 8.1.10   | Réinitialiser le score                                             | 22        |
| 8.1.11   | Traduction                                                         | 22        |
| <b>9</b> | <b>Conclusion</b>                                                  | <b>23</b> |
| 9.1      | Bilan du travail réalisé                                           | 23        |
| 9.2      | Perspectives d'évolution                                           | 24        |

# 1 Introduction

## 1.1 Présentation du sujet

Ce projet consiste à concevoir et développer une application mobile Android, codée uniquement en Kotlin, à partir de zéro, en respectant les principes fondamentaux d'ergonomie, d'utilisabilité et d'architecture logicielle enseignés lors des cours et des travaux pratiques.

Le sujet proposé consiste à réaliser un quizz interactif basé sur les personnages Amiibo issus de l'univers Nintendo. Pour cela, l'application doit interagir avec une API REST publique (<https://amiiboapi.com>) afin de récupérer dynamiquement des données sur les jeux et les personnages.

L'utilisateur est invité à sélectionner plusieurs séries de jeux, à partir desquelles l'application génère des questions sous forme d'images avec trois propositions de réponse. Le jeu doit intégrer une base de données locale (Realm) pour stocker les données, une gestion des scores, une navigation fluide entre les activités, ainsi que des éléments d'interface intuitifs et interactifs (menus, Toasts, swipe, musique de fond, etc.).

## 1.2 Objectifs du rapport

Ce rapport a pour but de retracer de façon claire et structurée toutes les étapes de conception et de développement de l'application. Il met en avant les choix faits en matière d'ergonomie, de modélisation des fonctionnalités et de programmation, en expliquant les raisons de ces décisions.

Au-delà de la simple description de l'application finale, le rapport raconte aussi le parcours suivi : les problèmes rencontrés, les solutions envisagées, les choix retenus et leurs justifications.

C'est un rapport complet, où chaque décision est argumentée. Il cherche à trouver un bon équilibre entre modélisation, ergonomie et développement, tout en prenant du recul sur le résultat : tests, points à améliorer, et retour sur l'expérience utilisateur.

## 1.3 Remarque

N'ayant pas étudié l'UML au semestre précédent, je vous remercie de prendre en compte le caractère urgent et autodidacte de ma formation en UML concernant le diagramme de classes et le diagramme de cas d'utilisation.

Dans le cadre du cours \*Application mobile et Ergonomie\*, dispensé en Licence 3 Informatique au CERI d'Avignon, ce projet individuel a pour objectif de mettre en application les concepts théoriques et pratiques abordés tout au long du semestre.

# 2 Contexte de travail

## 2.1 Présentation de l'organisation

Ce projet s'inscrit dans le cadre du cours \*Application mobile et Ergonomie\* dispensé en Licence 3 Informatique au sein du CERI (Centre d'Enseignement et de Recherche en Informatique) de l'Université d'Avignon. Le cours vise à familiariser les étudiants avec les principes de conception d'interfaces mobiles et les bonnes pratiques en matière d'ergonomie, tout en développant une application fonctionnelle.

## 2.2 Structure et fonctionnement

Le projet est réalisé de manière individuelle, tout en étant encadré par des enseignements réguliers et des retours pédagogiques. Les étudiants sont guidés tout au long du semestre

à travers des séances de cours, de travaux pratiques et des échanges avec l'enseignant, pour valider les étapes de conception et d'implémentation.

### 2.3 Méthodologies employées

Une approche itérative a été adoptée, avec des phases de conception, de développement, de test, puis de réajustement.

Cette méthode, bien que fonctionnelle, s'avère particulièrement chronophage lorsqu'on dispose de peu d'expérience avec Android Studio et le langage Kotlin.

L'accent est mis sur l'ergonomie de l'interface. La progression est documentée à chaque étape afin de garder une trace des décisions prises, des problèmes rencontrés et des solutions mises en œuvre.

Une dernière phase de refactorisation du code est à prévoir afin d'optimiser la qualité de la production.

## 3 Outils employés (logiciels et autres)

Pour le développement de l'application, plusieurs outils ont été utilisés afin d'assurer la qualité du projet et d'optimiser le processus de conception et de réalisation.

### 3.1 Environnement de développement intégré (IDE)

Le principal outil utilisé est **Android Studio**, l'IDE officiel pour le développement d'applications Android. Il permet une intégration fluide de Kotlin ainsi qu'une gestion optimisée de l'interface utilisateur.

Le projet a été testé directement sur un appareil mobile en raison de limitations matérielles empêchant l'utilisation d'un AVD (Android Virtual Device).

### 3.2 Langage de programmation

Le langage utilisé est **Kotlin**, recommandé par Google pour le développement Android. Il a été choisi pour sa simplicité, sa concision, et ses fonctionnalités modernes qui facilitent le développement tout en garantissant de bonnes performances.

### 3.3 Gestion de la base de données

Pour la gestion des données, **Realm** a été utilisé. Il s'agit d'une base de données mobile performante et facile à intégrer, idéale pour stocker localement les données issues de l'API, telles que les objets **AmiiboGame**, les données utilisateurs préférentielles, ainsi que les objets **Amiibo** et leurs états associés.

### 3.4 Outils de versioning

**Git** a été utilisé pour le contrôle de version, permettant de suivre l'évolution du code et de gérer les différentes versions du projet.

**GitHub** a servi à héberger le dépôt distant.

Suite à certains problèmes techniques liés à mon compte GitHub, je n'ai pas pu organiser le code dans plusieurs branches comme prévu.

### 3.5 Outils d'intelligence artificielle

Pour résoudre certains problèmes techniques et faciliter le débogage, j'ai utilisé des outils d'intelligence artificielle tels que **Copilot** et **ChatGPT**. Ces assistants m'ont permis d'obtenir des suggestions de code, d'identifier plus rapidement des erreurs et d'améliorer la qualité générale de l'application.

### 3.6 Outils de traitement d'image

J'ai utilisé **GIMP** pour le traitement d'image, notamment pour enlever les arrière-plans et ajuster les visuels selon les besoins. Les images ont été exportées en formats PNG ou SVG, puis intégrées dans les fichiers XML de l'application.

### 3.7 Utilisation de maquettage

J'ai également utilisé **Figma** pour créer une maquette de l'application, ce qui m'a aidé à concevoir une interface claire et fonctionnelle en amont du développement.

Cependant, l'application ne suit pas la maquette à la lettre, celle-ci ayant été réalisée avant une étude approfondie du sujet. Certains éléments ont dû être adaptés ou modifiés en cours de développement.

### 3.8 Tests et débogage

Pour les tests et le débogage, l'**émulateur Android** a été utilisé pour simuler différents appareils. Des tests manuels ont été menés afin de vérifier la fluidité de l'interface et la stabilité de l'application.

Des tests unitaires ont également été réalisés pour valider la gestion des données et les interactions avec la base de données Realm.

## 4 Modélisation (via un logiciel dédié, use case, diagramme de classes, etc.)

### 4.1 Logiciel UML

Pour modéliser les différentes parties du système, j'ai utilisé **StarUML**, un logiciel qui permet de créer des diagrammes UML tels que les diagrammes de classes, de cas d'utilisation ou encore de séquence. Cela a facilité la compréhension de l'architecture de l'application tout en permettant de la documenter précisément. La modélisation, en particulier celle du diagramme de classes, a nécessité un temps conséquent. Elle m'a permis de structurer de manière détaillée les entités et leurs relations dans l'application.

### 4.2 Diagramme de cas d'utilisation





## 5 Difficultés rencontrées

### 5.1 Problèmes techniques

#### 5.1.1 Compatibilité et intégration

L'intégration de certaines bibliothèques et la gestion des dépendances ont été des défis notables, en particulier lors de la configuration de Realm pour la gestion des données persistantes. Realm étant une base de données mobile performante, son intégration n'a pas été immédiate.

L'une des difficultés majeures résidait dans la gestion des requêtes asynchrones, qui ont parfois créé des incohérences dans les données. Les données n'étaient pas toujours correctement mises à jour ou synchronisées au niveau des checkboxes, ce qui a entraîné des comportements inattendus. Cela a nécessité de nombreux tests pour ajuster les requêtes et obtenir un fonctionnement fiable.

La gestion de ces appels asynchrones a donc exigé une attention particulière pour garantir une synchronisation des données fiable et fluide.

La gestion de la musique a également été un défi technique majeur. L'objectif était de maintenir l'état de la musique même lorsque l'application était mise en arrière-plan ou fermée.

Initialement, j'avais configuré un service dédié à la gestion de la musique, mais je rencontrais un problème récurrent où la musique se mettait en pause automatiquement lorsque l'application était quittée, ce qui n'était pas souhaité.

J'ai essayé plusieurs solutions pour contourner ce problème, mais elles ne répondaient pas toujours à mes attentes. Après plusieurs tests et ajustements, j'ai pu trouver une solution en utilisant des services en arrière-plan et en persistant l'état de la musique à l'aide de préférences partagées.

J'ai fait face à un problème au niveau de l'intégration de la gestion de thèmes. Comme nous avons la possibilité de revenir sur le HomeActivity avec la flèche arrière, nous sommes obligé de la garder en arrière plan.

Sauf que lorsqu'on modifie le thème depuis RewardActivity et qu'on retourne sur HomeActivity, il faut pour la mettre à jour, ce qui fait que nous voyons ce changement de couleurs au niveau du thème.

Je n'ai pas su comment recréer une activity depuis un retour avant d'être dessus.

De plus la charge de travail était tellement conséquente que cela m'a paru secondaire après plusieurs tentatives de résolutions.

#### 5.1.2 Performance et optimisation

La gestion des performances a été un autre aspect technique complexe à aborder, surtout lorsqu'il s'agissait de récupérer et d'afficher les données provenant de l'API REST.

Les requêtes envoyées étaient parfois lentes, ce qui nuisait à l'expérience utilisateur.

Les appels réseau étaient effectués de manière synchrone, ce qui bloquait l'interface pendant leur traitement.

Afin d'éviter les ralentissements, il a été nécessaire d'optimiser ces requêtes, notamment en les exécutant de manière asynchrone et en utilisant des techniques de mise en cache pour les données fréquemment sollicitées.

Cela a permis de réduire significativement le temps de chargement des informations.

La récupération et l'affichage d'un grand nombre d'objets 'AmiiboGame' ont également posé un défi. Afin d'éviter de surcharger la mémoire et d'accélérer le rendu des données, j'ai

mis en place une stratégie d'optimisation qui consiste à charger les éléments en local. Cela a permis de garantir une expérience fluide, même lorsque l'utilisateur accède à une liste d'objets importante. En plus de cela, j'ai optimisé la gestion de la base de données Realm pour éviter les accès redondants et pour réduire le nombre de lectures nécessaires.

## 5.2 Problèmes organisationnels

### 5.2.1 Communication interne

La communication autour des problématiques techniques a parfois été difficile, notamment lors de l'intégration de la gestion asynchrone dans Realm. Travailler avec des API asynchrones a exigé de nombreux ajustements pour que le flux de données soit cohérent et non bloqué. Les échanges entre les différentes parties du code n'étaient pas toujours évidents, surtout quand le traitement des données dépendait d'une tâche asynchrone. En particulier, lorsque le résultat d'une requête asynchrone était nécessaire pour poursuivre le traitement, il a fallu prendre en compte le fait que l'exécution du code ne suivait pas un ordre séquentiel simple.

Pour résoudre ce problème, j'ai utilisé des callbacks et des écouteurs ('listeners'), qui ont permis de contrôler le flux d'exécution du programme.

### 5.2.2 Gestion du temps de chargement

Les problèmes liés aux transitions asynchrones et à la gestion des données ont prolongé le temps de développement. En effet, la gestion des requêtes réseau, l'intégration de la base de données et la gestion des transitions entre activités ont exigé un temps considérable d'ajustements et de tests. Ce processus a également entraîné des retards dans la gestion du temps de développement global. En particulier, chaque nouvelle fonctionnalité impliquait des vérifications rigoureuses pour s'assurer qu'il n'y avait pas de régressions dans l'application.

L'une des tâches les plus chronophages a été d'optimiser l'interface pour qu'elle reste réactive tout en chargeant de nombreuses données en arrière-plan, ce qui a eu un impact direct sur le temps de développement global.

### 5.2.3 Synchronisation

L'un des problèmes les plus frustrants est survenu dans la gestion du 'RecyclerView', qui est un composant essentiel pour afficher la liste des 'AmiiboGame'. Lorsque les objets n'étaient pas complètement chargés dans la base de données, le 'RecyclerView' était quand même affiché, ce qui entraînait des erreurs visuelles et des incohérences dans l'interface utilisateur. Cela a donné lieu à des comportements erratiques, où certaines données n'étaient pas visibles ou bien mal affichées.

Pour résoudre ce problème, j'ai intégré un écouteur pour m'assurer que la 'MainActivity' ne se lançait qu'une fois le chargement complet des données terminé. Bien que cette approche ait résolu le problème, elle a également alourdi le code, et la gestion des états asynchrones dans ce contexte a encore nécessité quelques ajustements pour éviter des incohérences futures.

## 6 Solutions disponibles, solutions choisies et justifications (avantages et inconvénients)

### 6.1 Solutions disponibles pour MainActivity

Pour récupérer les 'gameSeries' disponibles depuis l'API REST (<https://amiiboapi.com>), j'ai utilisé Retrofit. J'ai intégré ces données dans un 'ConstraintLayout' contenant un 'RecyclerView', contrairement à la consigne qui proposait un 'ListView'.

#### 6.1.1 ListView

- Adapté à des listes simples avec peu d'éléments.
- Facile à mettre en place.
- Compatible avec les anciennes versions d'Android sans dépendances supplémentaires.

#### 6.1.2 RecyclerView

- Adapté à des listes complexes, avec plusieurs types d'éléments, des animations ou des interactions.
- Meilleures performances.
- Plus flexible (liste, grille, etc.).
- Recommandé pour des applications modernes et maintenables.

#### 6.1.3 Analyse comparative

J'ai choisi d'utiliser un 'RecyclerView', car c'est l'outil que nous avons utilisé en cours, ce qui m'a permis de l'adapter plus rapidement. Je comptais également y intégrer une fonctionnalité permettant d'afficher les 'Amiibo' d'une 'gameSeries' en maintenant un élément appuyé. Cette fonctionnalité n'a finalement pas été implémentée, mais elle justifiait initialement ce choix. Dans ce contexte, un 'ListView' aurait pu suffire.

### 6.2 Construction des activités

La consigne indiquait : *"Le traitement de la requête permettant de récupérer les 'gameSeries' de l'API Amiibo se fera dans le 'onCreate()' de la classe 'MainActivity'."*

#### 6.2.1 Redéfinir les bases

J'aurais pu redéfinir l'architecture et faire de 'MainActivity' une sorte de 'Home', tandis que le 'RecyclerView' aurait été dans une 'ListActivity'.

#### 6.2.2 S'adapter à vos consignes

J'ai préféré respecter la consigne, tout en l'adaptant à mon architecture :

- L'utilisateur arrive d'abord sur une activité de chargement ('LoadingActivity').
- Il est ensuite dirigé vers un menu ('HomeActivity').
- Enfin, il accède à 'MainActivity', qui contient la liste des 'gameSeries'.

Ainsi, je respecte la consigne : - Le 'RecyclerView' est bien dans 'MainActivity'. - Les requêtes API sont bien faites dans 'MainActivity'.

#### 6.2.3 Analyse comparative

Dans la première option, j'aurais pu faire mes appels API avant 'MainActivity', ce qui aurait réduit le temps de chargement visible. Dans l'option choisie, je respecte la structure imposée tout en permettant une optimisation implicite : ne pas masquer les lenteurs de requêtes en les déplaçant ailleurs, mais bien les optimiser directement.

### 6.3 Structure des classes : 'Amiibo', 'AmiiboGame', 'AmiiboHeader'

J'ai choisi d'utiliser une 'MutableList' dans 'AmiiboHeader' pour sa capacité à être modifiée dynamiquement, comme suggéré dans le sujet.

#### 6.3.1 Adaptation

J'ai ajouté un attribut booléen 'isSelected' dans la classe 'AmiiboGame'. Les classes 'Amiibo' et 'AmiiboGame' héritent de 'RealmObject'. J'ai préféré inclure tous les attributs des objets JSON dans mes classes Kotlin, mais uniquement ceux utiles dans les objets Realm, pour éviter d'alourdir la base inutilement. Cela laisse la porte ouverte à de futurs besoins.

#### 6.3.2 Respect de la consigne

J'aurais pu créer une classe 'AmiiboGameObject' intermédiaire, que 'AmiiboGame' aurait étendue. Cela aurait permis de placer l'attribut 'isSelected' dans cette classe, tout en gardant 'AmiiboGame' conforme à l'API.

#### 6.3.3 Analyse comparative

J'ai trouvé plus naturel et simple d'ajouter directement l'attribut dans 'AmiiboGame'. Créer une classe uniquement pour cela me semblait excessif. Inclure tous les champs dans Realm aurait été lourd, et ne pas les inclure aurait nécessité de tout anticiper.

### 6.4 Menus et barres d'actions

#### 6.4.1 Menu de 'MainActivity'

J'ai ajouté un item pour valider la sélection, un autre pour tout sélectionner/désélectionner, et une flèche de retour. Je souhaitais permettre à l'utilisateur de revenir facilement en arrière, depuis toutes les activités sauf 'HomeActivity'.

#### 6.4.2 Menu de 'GameActivity'

Je n'ai pas intégré le score dans le menu car je voulais le styliser en XML librement, ce qui n'est pas simple avec un item de menu. Cependant, le score fonctionne comme attendu, à l'exception du style de la valeur "0", qui reste cohérent avec le reste de l'interface.

#### 6.4.3 Menu et barres de 'HomeActivity', 'RewardActivity', 'HelpActivity'

Styliser un menu étant complexe, j'ai inséré des 'Layouts' dans les barres d'action pour mieux gérer l'espace, afficher des icônes, et le niveau du joueur. Je n'ai pas pu utiliser les 'BottomBars' de Google, en raison de la version d'Android et des dépendances du projet. J'ai donc utilisé un 'LinearLayout' pour positionner proprement les boutons et écouter les clics.

### 6.5 Récupération des Amiibo

#### 6.5.1 Traitement local

Utilisation de :

```
suspend fun getAmiiboByGameSeries(): Response<AmiiboResponse>
```

pour récupérer tous les Amiibo.

### 6.5.2 Requête spécifique

Utilisation de :

```
suspend fun getAmiiboByGameSeries(@Query("gameseries") gameSeries: String)
```

pour ne récupérer que ceux d'une série donnée.

### 6.5.3 Analyse comparative

J'ai d'abord opté pour une requête spécifique à partir des 'gameSeries' sélectionnées dans l'adapter. Mais cela provoquait des ralentissements (jusqu'à 20 secondes selon le cas). J'ai ensuite préféré tout récupérer en une seule requête et filtrer localement selon les 'gameSeries' sélectionnées. Cela améliore le temps de réponse, malgré une quantité de données plus importante au départ. J'ai conservé la sélection dans l'adapter, afin d'éviter des conflits de synchronisation.

## 6.6 Initialisation de la sélection dans le 'RecyclerView'

Par défaut, aucun 'AmiiboGame' n'est sélectionné. Cependant, si une partie a déjà été lancée, la dernière sélection est restaurée automatiquement à partir de la base Realm.

### 6.6.1 Analyse comparative

Cela permet, si l'utilisateur souhaite rejouer avec les mêmes jeux, de ne rien avoir à sélectionner à nouveau. Sinon, une sélection aléatoire de 4 'AmiiboGame' est proposée par défaut.

## 6.7 Bouton "tout sélectionner/désélectionner"

### 6.7.1 Approche intelligente

Le bouton pourrait adapter son comportement en fonction de l'état de la liste, avec un écouteur analysant le nombre de jeux sélectionnés.

### 6.7.2 Approche simple

Le bouton sélectionne ou désélectionne tout sans tenir compte de l'état actuel.

### 6.7.3 Analyse comparative

J'ai choisi l'approche simple. Je n'y voyais ni un réel intérêt ergonomique, ni une plus-value en termes de confort utilisateur. L'inconvénient est qu'il faut parfois cliquer deux fois pour tout désélectionner si tout est déjà sélectionné.

## 7 Utilisabilité (points liés à l'ergonomie traités, etc.)

Cette section décrit les choix ergonomiques et d'interface réalisés tout au long du développement de l'application, ainsi que les tests effectués pour garantir une expérience utilisateur optimale.

### 7.1 Navigation intuitive

La navigation a été conçue pour être fluide et agréable, en permettant à l'utilisateur de se déplacer facilement entre les différentes sections de l'application et en pouvant revenir à tout moment à HomeActivity.

J'ai en effet fait une entrave à la consigne. Je ne permet pas à l'utilisateur de revenir à MainActivity mais à HomeActivity. Car j'utilise HomeActivity comme la place centrale de l'application afin d'obtenir une centralisation des différentes activités ajoutées, encore une fois dans l'objectif d'avoir un fonctionnement proche d'un vrai jeu.

Ainsi j'ai disposé dans toutes les activités une flèche permettant de retourner à l'activité précédente.

Ainsi, s'il l'on ne dispose pas de la barre android l'application est utilisable.

J'ai décidé que lorsqu'on arriverait d'une activité sur HomeActivity on ne peut pourait plus y revenir avec un retour.

### 7.2 Interaction avec les éléments

Les menus sont clairement définie avec des icônes personnalisée et de taille convenable pour obtenir une harmonie visuelle avec une aspect bouton. Un fond gris permettant d'être distinguable avec les différentes couleurs de thèmes applicables. Les actions sont regroupées de manière logique.

J'ai également mis ma propre icone d'application.

Les couleurs sont gérés dans un xml qui choisi le thème selectionné pour appliquer les couleurs.

Tous les textes visibles pour l'utilisateur sont stockés dans les strings.xml.

### 7.3 Lors d'un partie

Je permets au joueur se swiper à gauche ou bien d'appuyer sur la flèche en bas à gauche pour entraîne une question sur le gameserie, et de façon similaire en swipant à droite ou avec la flèche en bas à droite pour une question sur le nom.

J'ai imposé un court délai entre chaque question pour rendre l'expérience plus agréable. J'ai également choisi une police de mon côté, la Jersey 25 donnant un aspect retro afin de trancher avec l'aspect peut-être trop moderne.

Si l'utilisateur fait une mauvaise partie et qu'il et que son score est négatif, il perdra en effet des points. Il peut même perte des niveaux et bloquer les icones. Cependant si son icone est une icone bloqué, il l'a conservera tout de même.

J'ai fait en sorte d'éviter le spam de réponses durant le quiz en désactivant les TextView et en les réactivant à chaque question.

L'utilisateur est soumis à un délai 1 secondes d'attentes choisi arbitrairement, entre chaque réponse, pour simuler une attente naturelle et stimuler son envie de jouer .

## 7.4 Affichage des données

L'affichage des informations, telles que la liste des AmiiboGame, a été pensé pour garantir une lisibilité optimale avec un effet d'ombre.

Une activité permettant de consulter les statistiques de l'utilisateur de manières détaillées et une petite explication avec les règles du jeu. Cette activité est accessible depuis le home ou bien depuis le quiz si l'utilisateur n'a pas pensé précédemment à consulter le fonctionnement du quiz.

J'ai décidé de forcer le mode portrait dans toutes les activités sauf **MainActivity** car le design n'a pas été fait en sorte de pouvoir s'adapter et être scrollable.

Lors d'un quiz à chaque réponse je fais apparaître un Toast pour signaler si la réponse est bonne ou mauvais et j'indique la bonne réponse en changeant la couleur de la case correcte en vert.

## 7.5 Tests d'ergonomie

Des tests utilisateurs ont été réalisés par des personnes n'ayant pas reçu d'explications sur l'application pour évaluer la facilité d'utilisation de l'application.

Ces tests ont permis d'ajuster certains éléments de l'interface afin de rendre l'application encore plus intuitive et agréable à utiliser. Notamment au niveau du visuel.

J'ai pu conclure des différents retour qu'il fallait clairement définir les questions dans les parties. Que 10 questions était convenable et que la progression était un peu rapide.

D'un point de vu factuel, le quiz est un jeu moyennement captivant, faire un progression rapide en débloquent facilement les icônes cachées permet d'entretenir le mystère.

Poussant un joueur à certes finir rapidement, mais surtout à finir le jeu.

J'ai fait en sorte d'éviter de spammer la validation des AmiiboGame, afin de ne pas faire plusieurs requêtes.

L'utilisateur est soumis à un délai 3 secondes d'attentes choisi arbitrairement, pendant l'ouverture de l'application pour le mettre dans l'esprit de l'application.

Il est également soucis à quelques courts délais pour rendre l'application plus naturelle et fluide.

## 7.6 Gestion des données

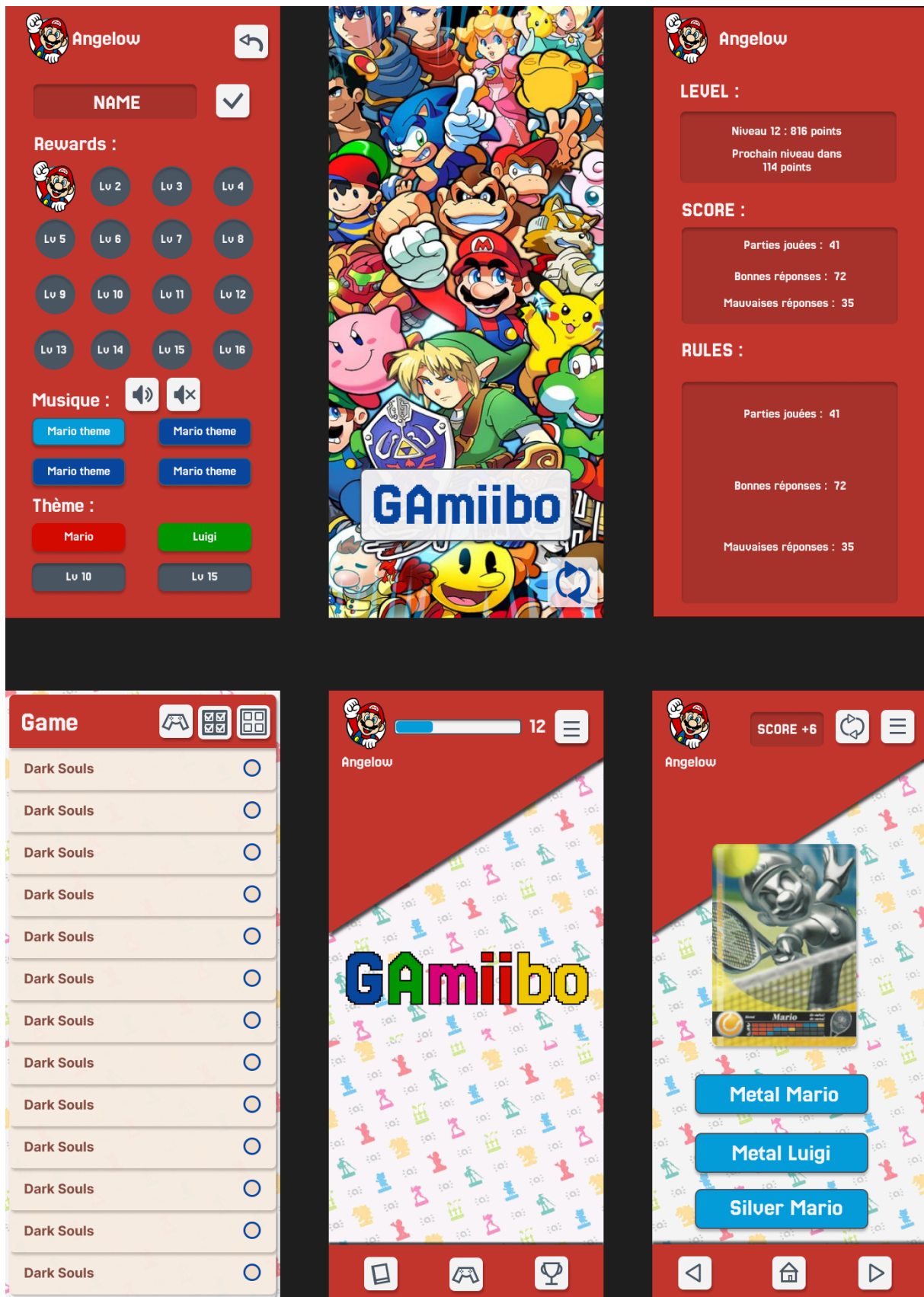
L'utilisateur à la possibilité lors d'un quiz de supprimer toutes les réponses précédentes. Les données de jeux sont stocké dans une base de donnée realm utilisateurs.

Je stock également les préférence de l'utilisateur.

Ainsi, il garde dans sa session, son thème, son nom, son nombre total de points, son nombre de victoire, de défaite etc.

L'utilisateur à la possibilité de totalement réinitialiser son compte en supprimant la base de donnée.

## 7.7 Maquette



## 7.8 Inspirations

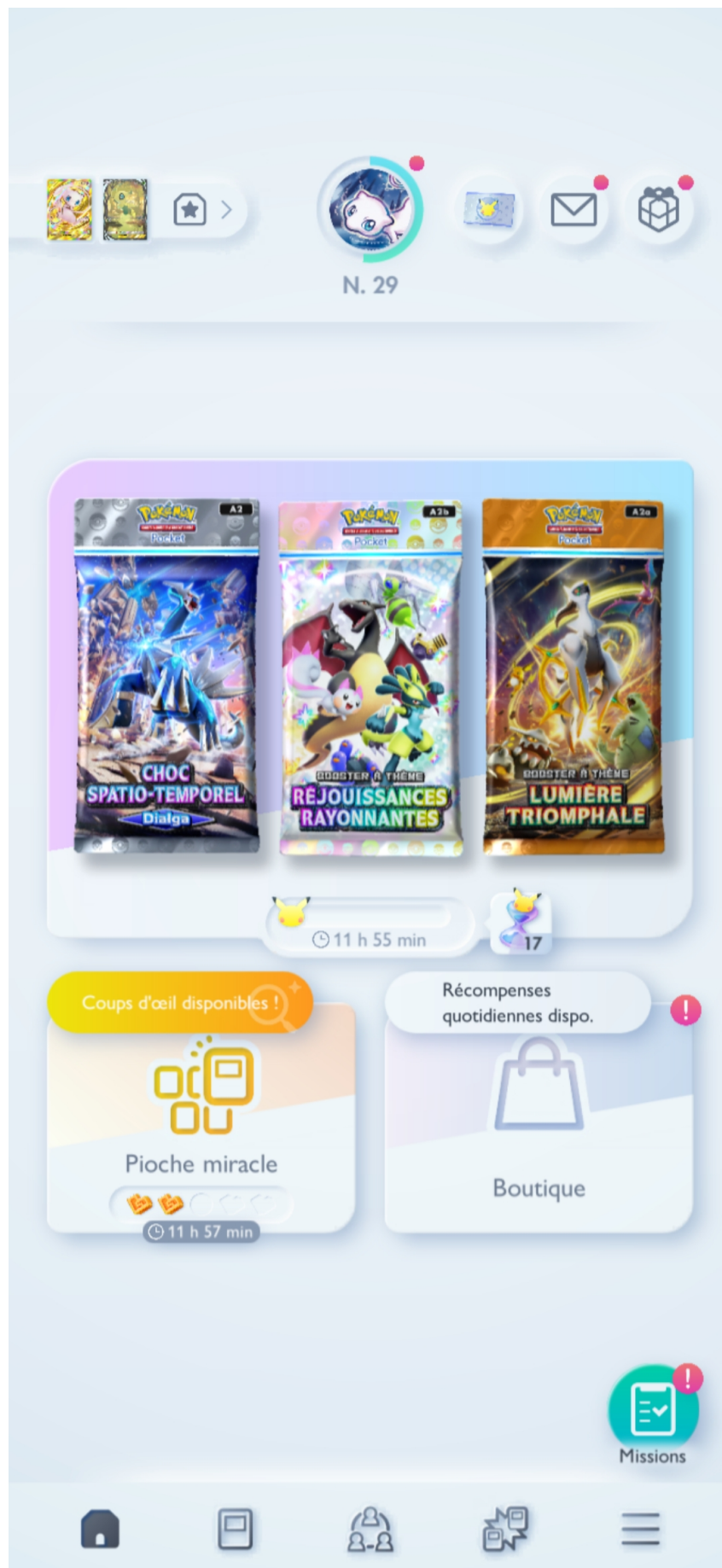
J'ai inclus dans l'application une référence à la **gameboy color**. En effet "Gamiibo" s'inspire des couleurs du texte "Gameboy".

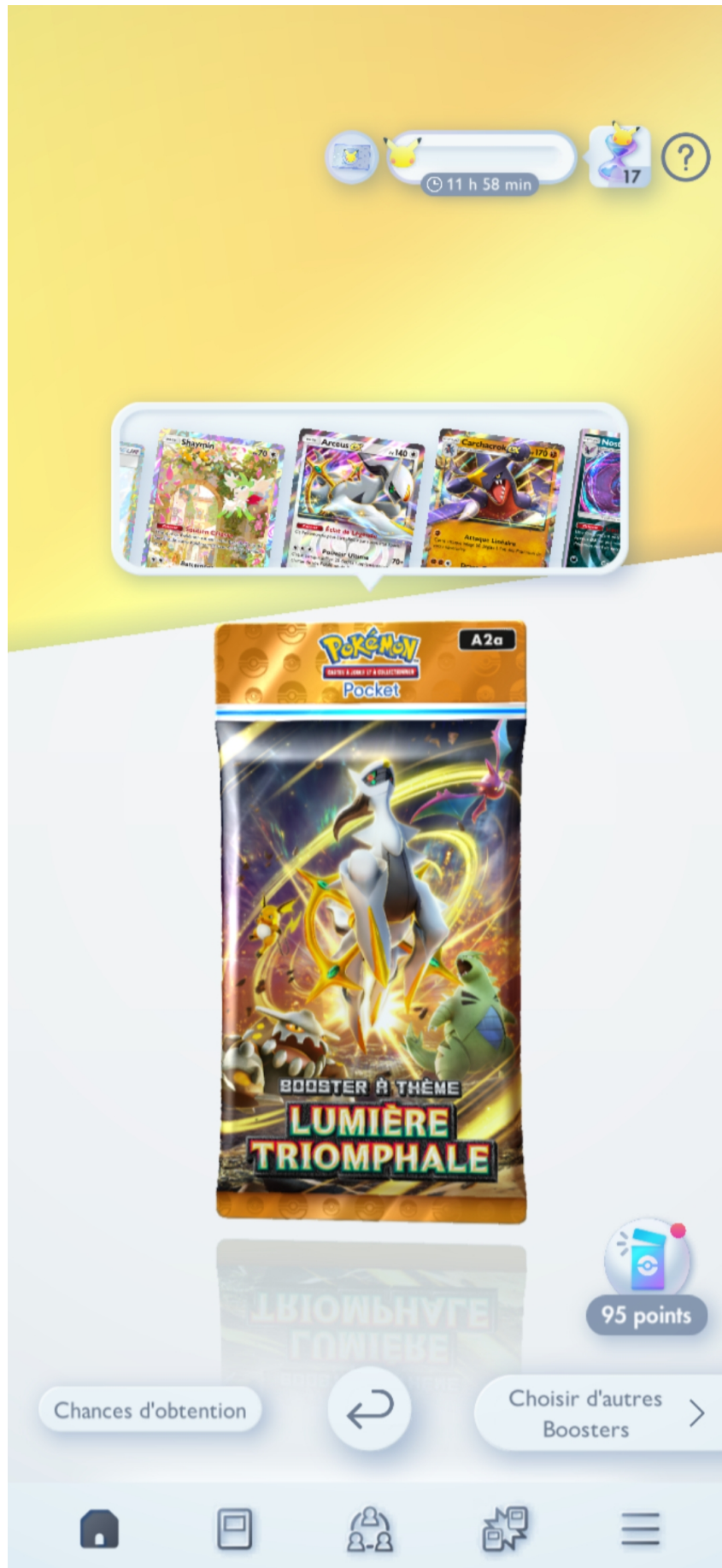
J'ai pu m'inspirer de différents jeux.

Les références principales sont :

- **Pokemon TCGP**
- **One Piece treasure Cruise**







## 8 Fonctionnalités développées

Dans cette section, nous présentons les différentes fonctionnalités développées et leur rôle dans l'application.

### 8.1 Présentation des fonctionnalités

#### 8.1.1 Changer de thème

L'application permet à l'utilisateur de changer le thème de l'interface. Cette fonctionnalité est persistante, ce qui signifie que le thème sélectionné est sauvegardé dans la base de données et appliqué lors des prochaines ouvertures de l'application.

#### 8.1.2 Gestion de la musique

L'utilisateur peut changer la musique, la stopper et la relancer selon ses préférences. L'état de la musique (en cours, arrêté) est également gardé en mémoire dans l'application grâce à la persistance des données dans la base de données. Ainsi, l'utilisateur retrouvera la musique dans le même état lors de son prochain usage de l'application.

#### 8.1.3 Changer d'icônes

Il est possible de changer les icônes de l'application, selon les préférences de l'utilisateur. Cette personnalisation est également persistée dans la base de données, ce qui permet à l'utilisateur de conserver ses choix entre les sessions.

#### 8.1.4 Choisir son nom d'utilisateur

L'utilisateur peut choisir et modifier son nom d'utilisateur. Cette donnée est stockée de manière persistante dans la base de données afin que l'utilisateur puisse conserver son nom entre les différentes utilisations de l'application.

#### 8.1.5 Suivi statistique

L'utilisateur peut suivre son avancée en regardant ses statistiques dans **HelpActivity** afin d'obtenir une expérience utilisateur d'avantage poussée.

#### 8.1.6 Une motivation pour jouer

L'utilisateur pourra débloquent une nouvelle icône en gagnant un niveau. Sachant qu'une partie comporte 10 questions, il peut donc gagner jusqu'à 20 points. De plus, le pallier d'un niveau est de 20 points, ainsi la progression peut-être rapide.

#### 8.1.7 Supprimer la base de données

Une fonctionnalité a été intégrée permettant à l'utilisateur de supprimer toute la base de données de l'application, réinitialisant ainsi toutes les préférences et données sauvegardées dont ses statistiques, nom, thèmes etc.

#### 8.1.8 Lancer un quiz

L'utilisateur peut démarrer un quiz à tout moment via une option dédiée dans l'application. Cette fonctionnalité permet à l'utilisateur de tester ses connaissances et d'interagir avec des questions personnalisées.

### 8.1.9 Choisir le type de Question

Le choix du type de question peut être effectué en utilisant un mécanisme de swipe ou en cliquant sur les flèches de navigation situées dans la barre de navigation en bas de l'écran. Cela permet à l'utilisateur de sélectionner facilement le type de questions qu'il souhaite rencontrer dans le quiz.

### 8.1.10 Réinitialiser le score

L'utilisateur peut réinitialiser son score depuis l'interface du quiz. Cela permet de tricher si on est mauvais.

### 8.1.11 Traduction

Tout les textes et toasts, hormis le nom d'utilisateur sont traduits en anglais et par défaut en français.

## 9 Conclusion

### 9.1 Bilan du travail réalisé

Ce projet a permis de créer une application ludique et interactive, centrée sur un quiz utilisant des Amiibo comme support. L'objectif était de combiner divertissement et personnalisation, tout en intégrant une gestion efficace des données et une expérience utilisateur fluide.

Tout au long du développement, plusieurs aspects ont été traités : l'ergonomie, la gestion des préférences utilisateur, l'intégration de la musique, ainsi que la persistance des données à l'aide de la base de données Realm.

Les choix de conception ont été guidés par la recherche d'une expérience immersive et cohérente. Par exemple, la centralisation des activités dans **HomeActivity** a permis d'offrir une structure claire et intuitive, mais a également impliqué certaines limitations ergonomiques, comme l'impossibilité de revenir à **MainActivity**.

Cependant, cette approche a renforcé l'idée d'une interface de type jeu, où chaque action mène à une nouvelle expérience sans confusion.

Les tests utilisateurs ont montré que l'application était généralement bien reçue, avec des améliorations à apporter principalement sur la progression des quiz et la définition plus précise des questions.

La gestion des données et la personnalisation via des icônes, thèmes et noms ont été des points forts de l'application, assurant une expérience unique pour chaque utilisateur.

Malgré les difficultés rencontrées dans le développement, notamment au niveau de la gestion des transitions et de la persistance des données, j'ai pris un réel plaisir à réaliser cette application.

Travailler sur l'aspect fonctionnel m'a permis de mettre en pratique des concepts de programmation, tandis que l'aspect esthétique m'a donné l'opportunité de concevoir une interface qui soit non seulement fonctionnelle mais aussi agréable à l'œil.

Ce mélange entre technique et design a été particulièrement enrichissant et m'a permis de progresser dans des domaines variés.

## 9.2 Perspectives d'évolution

Bien que l'application réponde aux besoins principaux de l'utilisateur, plusieurs pistes d'amélioration et d'extension existent pour enrichir l'expérience et accroître l'interactivité :

- **Ajouter un mode de jeu aléatoire** : La possibilité de choisir des jeux de manière aléatoire depuis **MainActivity** permettrait d'apporter plus de diversité et de surprise, enrichissant ainsi la rejouabilité de l'application.
- **Améliorer l'adaptabilité des boutons** : En fonction de l'état du **RecyclerView**, les boutons pourraient se réorganiser ou s'ajuster de manière plus dynamique. Cette évolution rendrait l'interface plus réactive et intuitive.
- **Optimisation responsive** : Rendre l'application responsive en permettant l'utilisation en mode paysage et en optimisant l'interface pour différentes tailles d'écran garantirait une meilleure accessibilité, notamment pour les utilisateurs de tablettes ou d'appareils plus grands.
- **Amélioration des performances et de la gestion des données** : Bien que Realm fonctionne de manière efficace pour la persistance des données, une analyse plus approfondie des performances, notamment pour le stockage et la gestion des données utilisateurs en cas de sessions longues ou de très nombreuses réponses dans le quiz, pourrait offrir un gain de fluidité.
- **Barre de progression** : J'aurais voulu, comme sur ma maquette, ajouter une barre de progression en fonction du nombre de points pour rendre cela plus esthétique.
- **Nombre de questions** : J'aurais voulu, faire choisir à l'utilisateur son nombre de questions.

En somme, ce projet a permis de développer une base solide pour une application de quiz enrichissante, avec une personnalisation poussée et une interface agréable. Les évolutions futures permettront de renforcer l'expérience utilisateur en offrant davantage de flexibilité, de réactivité et de contenu.